TinyML-Based Lightweight Deep Learning Model for Human Activity Recognition on Edge Devices

Param Sharma[1][0009-0005-7308-6401], Pranay Agarwal[1][0009-0009-6879-4074], and Jitendra Goyal[2][0000-0003-3147-8237]

[1] Dept. of CSE, Birla Institute of Technology Mesra, Ranchi, India
[2] Dept. of CSE, Assistant Professor, Birla Institute of Technology Mesra, Ranchi, India
btech25035.22@bitmesra.ac.in

Abstract. Human activity recognition (HAR) using smartphone sensors is an important component in health monitoring, fitness tracking, and assistive technologies. However, it remains a big challenge to deploy deep learning models for HAR on small and resource-constrained devices because of their high computational and memory requirements. The proposed study investigates how small and efficient neural networks can accurately recognize human activities from smartphone sensor data in real time. Using the UCI Human Activity Recognition dataset, three one-dimensional deep learning architectures—NormalCNN1D, MobileNet1D, and a CNN-BiLSTM hybrid are trained and compared for classifying six common daily activities, such as walking, standing, and sitting. To enable deployment on embedded hardware, the best-performing model is further optimized through pruning and quantization to minimize memory usage while preserving accuracy. All models achieved strong classification performance, with MobileNet1D outperforming others by reaching 92.77% accuracy and maintaining a balanced precision-recall trade-off. After quantization, the model retains nearly identical accuracy while reducing its size to under 1.5 MB. These results show how deep learning models can be effectively adapted for real-time, on-device activity recognition, advancing the development of energy-efficient and scalable TinyML applications for edge computing environments. The paper demonstrates a lightweight deep neural network that can classify six daily activities from smartphone sensor data and then shows how to compress it enough to deploy on an edge device. The proposed study focuses on two objectives: achieving high classification accuracy and reducing the model size, while retaining performance.

Keywords: TinyML · Human Activity Recognition · 1D CNN · MobileNet1D · Edge AI · Model Compression · BiLSTM Hybrid

1 Introduction

Phones and wearable sensors collect motion data from users' daily movements like walking, sitting, climbing stairs, or simply standing still [19]. Turning those

motion signals into meaningful labels is the main idea behind Human Activity Recognition (HAR) [7]. It's applied in areas such as fitness tracking and health monitoring, to gesture control and fall detection.

However, practical deployment faces issues related to speed and scalability. A model that runs perfectly on a GPU in the lab doesn't always fit on a wristband or microcontroller. That's where TinyML helps us—the process of optimizing deep learning models so they can run directly on constrained hardware without cloud dependence [1,4,14]. Recent studies [1,4] emphasize that TinyML addresses limitations of cloud-centric AI by enabling on-device inference near the data source, resulting in reduced latency, enhanced privacy, and enabling continuous operation even without internet connectivity. Through the use of compact neural models and hardware-specific design, TinyML allows real-time analysis and decision-making directly on embedded platforms. This capability is particularly beneficial for use cases such as Human Activity Recognition, which demand rapid response and minimal energy consumption [6].

2 Literature Survey

Human Activity Recognition (HAR) has changed a lot in the last twenty years. It has gone from basic statistical methods to more advanced deep learning pipelines. In the beginning, most HAR systems used a lot of hand-engineered features taken from readings from an accelerometer or gyroscope. Researchers usually used mean, variance, signal magnitude area, FFT coefficients, or correlation-based descriptors to show how things move [2, 26]. Earlier, people often used models like SVMs, k-NN, Random Forests, and HMMs for HAR tasks. They did a decent job when things were controlled in the lab, but not so much in real-life situations—there's just too much noise, and people don't always act the same way. Plus, the fact that time matters a lot makes things tricky [10].

Things changed a lot around 2015 to 2017 when deep learning took off in this field. 1D convolutional models began to outperform older methods because they could just learn from the raw signals, without much manual feature design [12,17]. Later, hybrid models like CNN-LSTM and CNN-GRU popped up. These mixed together the strengths of convolutional layers (which spot patterns) and recurrent layers (which remember sequences) [8]. They were great for accuracy—but the catch was, they needed a lot of memory and computing power, so it wasn't practical to run them on phones or other little devices [5].

In the last few years, researchers have been more interested in deep learning that uses fewer resources. Lightweight architectures like MobileNet, ShuffleNet, and SqueezeNet showed that depthwise-separable convolutions and pointwise operations can cut down on the number of parameters needed while still keeping accuracy high [1,4]. At the same time, TinyML came about as a separate field that focuses on real-time inference on microcontrollers and low-power chips. Research investigated pruning, quantization, low-rank decomposition, and knowledge distillation to compress models while minimally impacting performance [9, 11, 13]. These techniques have demonstrated potential in image and speech domains;

however, their application to sensor-based Human Activity Recognition (HAR) remains constrained and predominantly experimental [3, 15, 20].

Recent advances have also explored transformer-based and multimodal approaches for HAR, improving robustness in more complex scenarios [16, 23]. A number of applied TinyML works have shown promising prototypes and real-device evaluations [12, 15, 20]. Taken together, these studies show clear progress but also stress the need for complete workflows that take a model from training to on-device deployment — which is what our work aims to demonstrate practically [18, 24].

In this context, our research enhances the current body of knowledge in three significant ways. First, instead of just using big hybrid models, we systematically compare three carefully chosen 1D deep learning architectures: a baseline CNN, a lightweight MobileNet1D, and a CNN-BiLSTM hybrid. We show how the accuracy–efficiency trade-off works between them. Second, we use post-training INT8 quantization to evaluate how well HAR models retain accuracy when optimized for edge deployment [9, 11, 13]. Third, we show how lightweight models can perform well in real time on embedded platforms while still being competitive with larger architectures by testing the compressed networks on important performance metrics and ROC/PR curves. So, our study not only summarizes what has already been done, but it also moves things forward by showing a complete, deployment-oriented TinyML-HAR workflow [25].

Along with HAR-focused deep learning approaches, several studies focusing on optimisation in related domains have used the lightweight and efficiency-oriented model design under resource constraints. For instance, recently, authors working in video and medical image watermarking have made use of meta-heuristic and multi-objective optimisation approaches in order to get better computational efficiency and improve robustness while maintaining performance [21, 22]. All these studies show the importance of accuracy-efficiency trade-offs in environments having resource constraints and align with the aim of this work, where TinyML-based HAR models are balancing accuracy with deployment efficiency.

3 Model Architecture

In Experimental Analysis, we are using three different models, which are as follows:

3.1 NormalCNN1D

The baseline CNN consists of two convolutional layers (64 and 128 filters) [2]. The next two layers are a max-pooling layer and a dropout layer to prevent overfitting. The output feature maps are flattened and passed through two fully connected layers [17]. The final layer applies a softmax activation across six activity classes. Overall, the model serves as a simple and effective baseline for time-series classification [17].

3.2 MobileNet1D

The MobileNet1D model employs depthwise-separable convolutions, where each input channel is processed by a single filter and then combined through pointwise convolution [4]. This approach reduces the number of learnable parameters and lowers memory usage. A global average pooling layer replaces the flattening step, contributing to an efficient structure [1]. Although the parameter count is reduced, MobileNet1D maintains high representational capability and converges faster than the baseline model [12].

3.3 CNN + BiLSTM Hybrid

In the hybrid design, for extracting the spatial features, convolutional layers are used, which are then processed by a bidirectional LSTM to learn temporal patterns in both directions [8, 17]. The combinations help recognize subtle transitions between activities, although it come with extra computational cost [7].

3.4 Proposed TinyML-HAR Workflow

Algorithm 1 End-to-End TinyML-HAR Pipeline

Require: Raw accelerometer and gyroscope signals
Ensure: Activity class label for each time window
1: **Step 1:** Collect raw motion signals from smartphone sensors.
2: **Step 2:** Apply noise filtering, normalization, and segmentation into fixed 2.56-second overlapping windows.
3: **Step 3:** Reshape feature vectors into $(samples, 561, 1)$ format for 1D CNN processing.
4: **Step 4:** Train three deep learning models: NormalCNN1D, MobileNet1D, and CNN-BiLSTM on the prepared dataset.
5: **Step 5:** Evaluate models using accuracy, precision, recall, F1-score, confusion matrix, ROC and PR-curve metrics.
6: **Step 6:** Apply weight pruning to remove low-magnitude connections and redundant filters, reducing active parameters while preserving accuracy.
7: **Step 7:** Fine-tune the pruned model and export the compressed network.
8: **Step 8:** Perform post-training INT8 quantization to further reduce model size and enable efficient TinyML deployment.
9: **Step 9:** Convert the optimized model to a TFLite edge-compatible format and deploy it on microcontroller-class hardware for real-time inference.

The workflow provides a complete pipeline from sensor data acquisition to on-device HAR execution, making sure that model accuracy is preserved along with the computational and memory constraints of TinyML platforms.

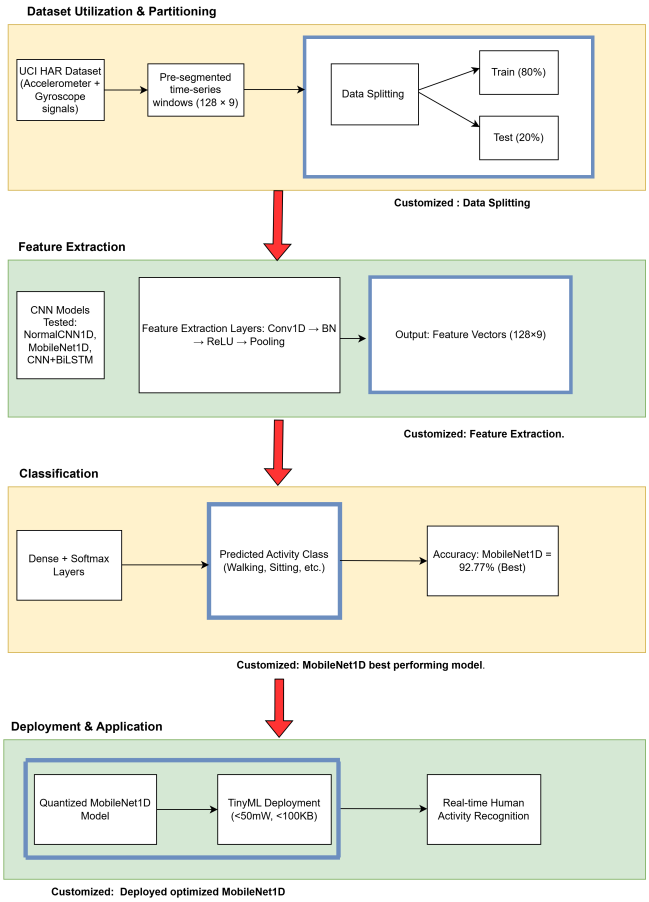


Fig. 1: Architectural Layout of Evaluated Models

4 Experimental Setup

4.1 Dataset and Preprocessing

The current study utilizes the UCI Human Activity Recognition dataset collected via Smartphones [19]. The data is gathered from 30 different individuals engaged in six daily activities, namely walking, walking upstairs, walking downstairs, sitting, standing, and lying down [19, 26]. Participants carry a waist-mounted smartphone equipped with accelerometer and gyroscope sensors that record motion signals along with 3-axis linear acceleration, and angular velocity at 50 Hz [19]. The continuous signal stream is divided into overlapping segments of 2.56 seconds (128 readings) with a 50% overlap. Every window corresponds to a single labelled activity [17, 19].

The dataset includes 561 features per sample, extracted from both time and frequency domains [17]. Input vectors were normalized to zero mean and unit variance, then reshaped into (samples, 561, 1) for the 1D CNNs. Activity labels are converted into six one-hot encoded categories [19].

4.2 Workflow

Figure 2 summarizes the complete workflow for human activity recognition using smartphone sensor data. The process begins with collecting raw signals from devices, followed by preprocessing steps such as noise filtering, normalization, and segmentation into fixed time windows. Features are then extracted through both statistical methods and deep learning layers, producing vectors for classification. Three neural models, NormalCNN1D, MobileNet1D, and CNN+BiLSTM, are trained on these features to predict six different activity classes. The final stage demonstrates deployment on TinyML hardware like Arduino or ESP32, where the optimized model delivers real-time inference with low latency and power usage, suitable for on-device applications [12, 15].

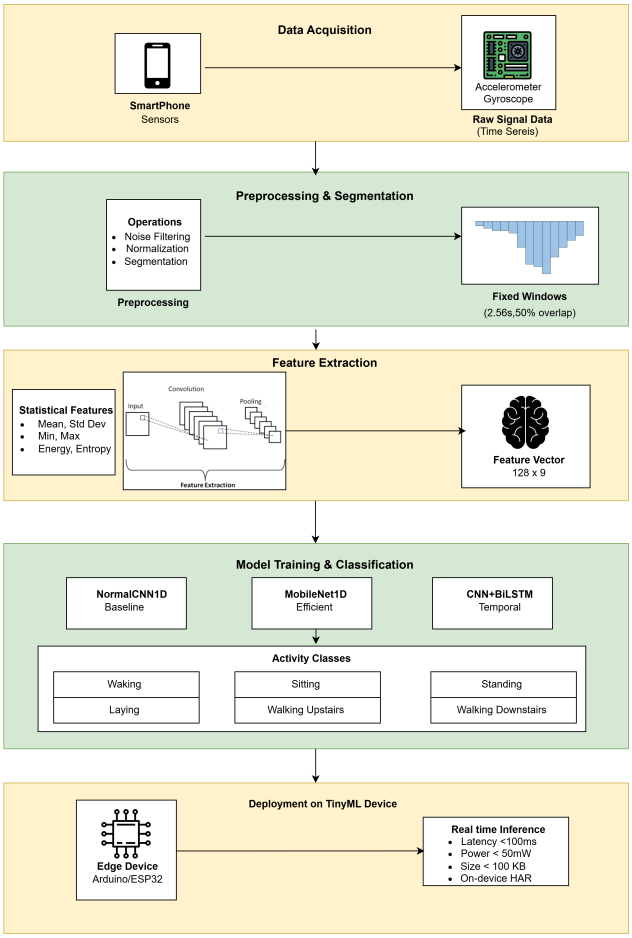


Fig. 2: Workflow Proposed TinyML-HAR System.

5 Experimental Results

Once the baseline CNN reached stable accuracy, it was optimized for deployment using TensorFlow Model Optimization Toolkit [25].

5.1 Pruning

Low-magnitude weights and redundant filters are removed, reducing the number of active parameters by nearly 60%. The pruned model retains almost the same accuracy while requiring significantly less memory.

5.2 Quantization

Weights and activations are converted from 32-bit floating-point representations to 8-bit integer representations using post-training quantization. This significantly reduces memory footprint and improves execution efficiency on low-power embedded hardware without requiring model retraining. The quantized model achieves a 3–4× speed-up on ARM-based devices while retaining accuracy within a marginal performance drop.

5.3 Knowledge Distillation

As part of the TinyML compression pipeline, the largest CNN model acts as a teacher network, and a smaller student network is trained to learn from the softened output probabilities of the teacher model. This enables the student network to approximate similar decision boundaries even after heavy compression, improving generalization performance while maintaining lightweight model complexity.

Table 1: Compression among Models.

Model	Size (MB)	Accuracy (%)	Reduction
Baseline CNN (FP32)	5.6	92.03	—
Pruned CNN	2.3	91.78	59%
Quantized CNN (INT8)	1.4	91.62	75%

5.4 Performance Metrics

Table 2 gives a pretty good look at how the three models compare side by side. NormalCNN1D held up pretty well with solid scores all around, so even a basic convolutional setup does the job for most activity types. MobileNet1D gave slightly better results across the board—it was better at not making mistakes and also didn't miss when something actually happened, so it ended up with the best overall accuracy.

CNN+BiLSTM did okay, mostly thanks to being able to follow patterns over time. But honestly, the extra complexity didn't really pay off with better scores on this dataset.

All in all, MobileNet1D was the standout. It's reliable, accurate, and doesn't need much power, which makes it the best choice of the bunch [6, 10].

5.5 Quantitative Result Interpretation

After the comparison using various parameters (accuracy, precision, recall, f1score), the best performing model obtained was MobileNet1D due to its depthwise separable convolution blocks, which enabled better feature extraction while

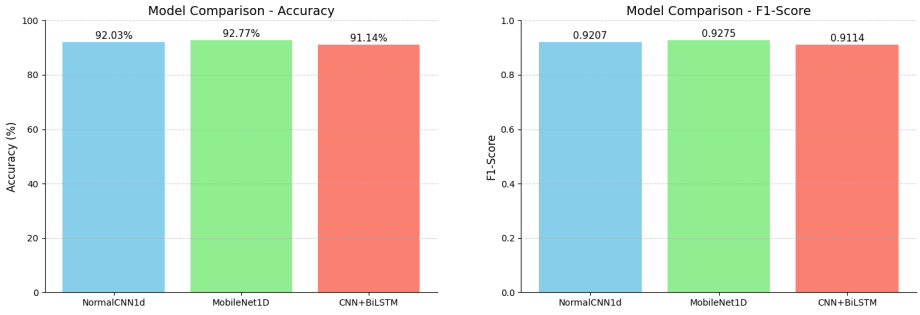
Table 2: Performance metrics of evaluated HAR models.

Model	Accuracy (%)	Precision	Recall	F1-Score
NormalCNN1D	92.03	0.9205	0.9209	0.9207
MobileNet1D	92.77	0.9270	0.9280	0.9275
CNN + BiLSTM	91.14	0.9110	0.9119	0.9114

significantly reducing the parameter count. Though the temporal sequence modelling benefited the CNN-BiLSTM model but the recurrent layers added higher complexity, failing to give an equivalent accuracy gain for the dataset. The baseline NormalCNN1D performs competitively but lacks the representational efficiency of MobileNet1D. All these results showed that by using the lightweight convolutional architectures, better results were obtained when optimized for resource constrained HAR applications in comparision to the heavier hybrid networks. This reinforces the suitability of MobileNet1D for TinyML deployment.

5.6 Visual Comparisons

Bar charts from Figures 3–3d show accuracy, precision, recall, and F1-score generated for visualizing the results. Accuracy is shown in percentage form, with precision, recall, and F1-score reported as decimal values. All charts demonstrate that MobileNet1D maintained a modest yet steady lead over the other models.



(a) Accuracy across the three models.

(b) F1-Score across the three models.

(c) Precision across the three models.

(d) Recall across the three models.

Fig. 3: Bar charts comparing key metrics across the evaluated models: (a) Accuracy, (b) F1-score, (c) Precision, and (d) Recall.

5.7 ROC Curve Analysis

Multiclass ROC curves were generated for each of the six activity categories. MobileNet1D had the highest area under the curve (AUC) values for most activity classes, indicating superior class separation [10, 20].

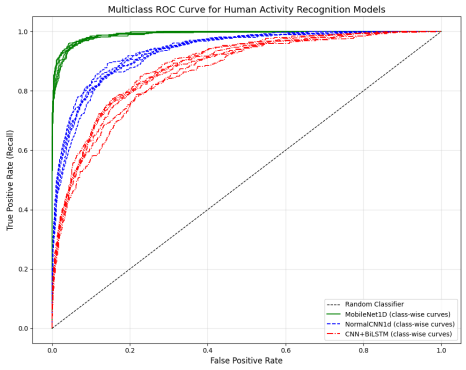


Fig. 4: Class-wise ROC comparison of HAR models.

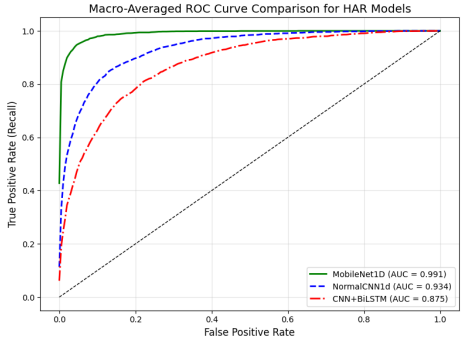


Fig. 5: Macro-averaged ROC comparison of HAR models.

5.8 Precision-Recall Analysis

Macro-average PR curves gave further insight. MobileNet1D achieved the highest average precision (AP), indicating fewer false positives and greater confidence in its predictions. This balance between precision and recall is important for real-world activity monitoring, where false triggers can drain power or annoy users [3, 15].

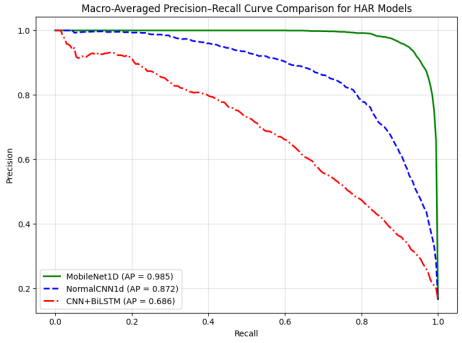


Fig. 6: Macro-averaged Precision–Recall curve.

6 Conclusion and Future Scope

Our tests show you can get really accurate results in human activity recognition using smaller, efficient deep learning models—no need for huge, complex systems. For example, MobileNet1D stood out, achieving 92.77% accuracy while keeping the model size small. That made it the best mix of accuracy and efficiency out of the three models we tried.

We also noticed that both convolutional networks and models that track sequences, like CNN+BiLSTM, did a good job handling motion data gathered over time. Even when we shrank the models for more limited devices, their performance didn't drop much at all. After post-training quantization, the model shrank to almost one-fourth of its initial size, and pruning eliminated redundant weights by nearly 60% while maintaining comparable accuracy. All of this really supports the idea that you don't always need massive, complicated networks for sensor-based human activity recognition. With some smart design choices, even tiny models can be surprisingly effective. TinyML, in particular, looks like a great way to power on-device HAR systems that need to be fast, battery-friendly, and keep user data private. The results also demonstrate how post-training optimization combined with structured model design (e.g., depthwise separable convolutions) can yield highly effective models appropriate for microcontroller-based platforms, smartphones, and wearables.

We intend to measure real-world latency, energy consumption, and responsiveness in subsequent work by directly validating the compressed models on embedded hardware. Looking ahead, there's a lot to try: using transformers customized for microcontrollers, adding federated learning to make activity recognition more tailored to each person, or updating the system so it can learn on the fly as it gets more data . If we can do those things, these HAR systems could end up being even more useful and reliable for everyone.

In the future, experiments could include real-world evaluation on embedded platforms, optimization for energy efficiency, and integration of federated learning to enable private, user-adaptive updates.

References

1. Abadade, Y., Temouden, A., Bamoumen, H., Benamar, N., Chtouki, Y., Hafid, A.S.: A comprehensive survey on tinyml. IEEE Access **11**, 96892–96922 (2023). https://doi.org/10.1109/ACCESS.2023.3294111
2. Aggarwal, J.K., Xia, L.: Human activity recognition from 3d data: A review. Pattern Recognition Letters **48**, 70–80 (2014). https://doi.org/10.1016/j.patrec.2014.04.011
3. Alajlan, N.N., Ibrahim, D.M.: Ddd tinyml: A tinyml-based driver drowsiness detection model using deep learning. Sensors **23**(12), 5696 (2023). https://doi.org/10.3390/s23125696
4. Capogrosso, L., Cunico, F., Cheng, D.S., Fummi, F., Cristani, M.: A machine learning-oriented survey on tiny machine learning. IEEE Access **12**, 23406–23426 (2024). https://doi.org/10.1109/ACCESS.2024.3365349
5. Cheng, X., Zhang, L., Tang, Y., Liu, Y., Wu, H., He, J.: Real-time human activity recognition using conditionally parametrized convolutions on mobile and wearable devices. arXiv (2020). https://doi.org/10.48550/arxiv.2006.03259
6. Contoli, C., Freschi, V., Lattanzi, E.: Energy-aware human activity recognition for wearable devices: A comprehensive review. Pervasive and Mobile Computing **104**, 101976 (2024). https://doi.org/10.1016/j.pmcj.2024.101976
7. Demrozi, F., Pravadelli, G., Bihorac, A., Rashidi, P.: Human activity recognition using inertial, physiological and environmental sensors: A comprehensive survey. IEEE Access **8**, 210816–210836 (2020). https://doi.org/10.1109/ACCESS.2020.3037715
8. Duan, F., Zhu, T., Wang, J., Chen, L., Ning, H., Wan, Y.: A multi-task deep learning approach for sensor-based human activity recognition and segmentation. arXiv (2023). https://doi.org/10.48550/arxiv.2303.11100
9. Frankle, J., Carbin, M.: The lottery ticket hypothesis: finding sparse, trainable neural networks. arXiv (2018). https://doi.org/10.48550/arxiv.1803.03635
10. Garcia-Gonzalez, D., Rivero, D., Fernandez-Blanco, E., Luaces, M.R.: Deep learning models for real-life human activity recognition from smartphone sensor data. Internet of Things **24**, 100925 (2023). https://doi.org/10.1016/j.iot.2023.100925
11. Gong, Y., Liu, L., Yang, M., Bourdev, L.D.: Compressing deep convolutional networks using vector quantization. arXiv (2014). https://doi.org/10.48550/arxiv.1412.6115
12. Gupta, S., Jain, S., Roy, B., Deb, A.: A tinyml approach to human activity recognition. In: Journal of Physics: Conference Series. vol. 2273, p. 012025 (2022). https://doi.org/10.1088/1742-6596/2273/1/012025
13. Han, S., Mao, H., Dally, W.J.: Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. arXiv (2015). https://doi.org/10.48550/arxiv.1510.00149
14. Heydari, S., Mahmoud, Q.H.: Tiny machine learning and on-device inference: A survey of applications, challenges, and future directions. Sensors **25**(10), 3191 (2025). https://doi.org/10.3390/s25103191
15. Im, H., Lee, S.: Tinyml-based intrusion detection system for in-vehicle network using convolutional neural network on embedded devices. IEEE Embedded Systems Letters (2024). https://doi.org/10.1109/LES.2024.3475470
16. Khan, S.U., et al.: Multimodal feature fusion for human activity recognition using human-centric temporal transformer. Engineering Applications of Artificial Intelligence **160**, 111844 (2025). https://doi.org/10.1016/j.engappai.2025.111844

17. Ordóñez, F.J., Roggen, D.: Deep convolutional and LSTM recurrent neural networks for multimodal wearable activity recognition. Sensors **16**(1), 115 (2016). https://doi.org/10.3390/s16010115
18. Ren, H., Anicic, D., Runkler, T.: How to manage tiny machine learning at scale: An industrial perspective. arXiv (2022). https://doi.org/10.48550/arxiv.2202.09113
19. Reyes-Ortiz, J.L., Anguita, D., Ghio, A., Oneto, L., Parra, X.: Human activity recognition using smartphones dataset. UCI Machine Learning Repository (2012). https://doi.org/10.24432/C54S4K
20. Signoretti, G., Silva, M., Andrade, P., Silva, I., Sisinni, E., Ferrari, P.: An evolving tinyml compression algorithm for iot environments based on data eccentricity. Sensors **21**(12), 4153 (2021). https://doi.org/10.3390/s21124153
21. Singh, R., Pal, R., Joshi, D.: Optimal frame selection-based watermarking using a meta-heuristic algorithm for securing video content. Computers & Electrical Engineering **121**, 109857 (2025). https://doi.org/10.1016/j.compeleceng.2024.109857
22. Singh, R., Pal, R., Mittal, H., Joshi, D.: Multi-objective optimization-based medical image watermarking scheme for securing patient records. Computers & Electrical Engineering **118**, 109303 (2024). https://doi.org/10.1016/j.compeleceng.2024.109303
23. Suh, S., Rey, V.F., Lukowicz, P.: Tasked: Transformer-based adversarial learning for human activity recognition using wearable sensors via self-knowledge distillation. arXiv (2022). https://doi.org/10.48550/arxiv.2209.09092
24. Suwannaphong, T., Jovan, F., Craddock, I., McConville, R.: Optimising tinyml with quantization and distillation of transformer and mamba models for indoor localisation on edge devices (2025). https://doi.org/10.1038/s41598-025-94205-9
25. Team, T.: Tensorflow model optimization toolkit. https://www.tensorflow.org/model_optimization, accessed 2025
26. Wang, Y., Cang, S., Yu, H.: A survey on wearable sensor modality centred human activity recognition in health care. Expert Systems With Applications **137**, 167–190 (2019). https://doi.org/10.1016/j.eswa.2019.04.057